Detection-Driven Layout Analysis and Template Generation for Korean Card-News, with a Sequential Analysis of Multi-Page Decks

Seungju Lee Hyunmin Han Junhyun Park
Computer Vision & Time-Series Project
cardnews-cv

Abstract

Card-news—short, image-based slide decks widely used on Korean social media—combine photographic backgrounds with carefully placed Korean typography. We study the problem of (i) **analyzing** the layout of existing card-news and (ii) **generating** new, production-quality cards. We build a curated corpus of **687** Korean cards organized into **66 multi-page decks**, and bootstrap element annotations (title / body) with a high-recall OCR pipeline and line-to-block merging. A YOLOv8 detector fine-tuned on these pseudo-labels reaches **mAP@50–95 = 0.718** (mAP@50 = 0.854); transfer learning is decisive (scratch: 0.502). We then compare two layout-generation routes: a research baseline that adapts the DS-GAN of PosterLayout [1], and a **detection-driven template engine** that extracts real layouts and re-renders crisp vector text with a designer Korean typeface. The template engine is markedly more reliable and legible for deployment, where GAN-rendered text is unusable. Finally, treating each deck as an ordered sequence, we present a **time-series analysis** of layout dynamics across page position over 66 decks, revealing a consistent "cover vs. interior" structure (covers are darker, more saturated and less edge-dense). Code, data tooling, and figures are released.

1 Introduction

Korean *card-news* (*kadeunyuseu*) are multi-page visual posts that compress an article into a sequence of designed slides. Producing them well requires content-aware placement of design elements—**title**, **body**, **logo**, **underlay**—so that text stays readable, avoids salient image regions, and follows a consistent visual rhythm across pages. This is exactly the *content-aware visual–textual layout* problem formalized by PosterLayout [1], but specialized to (a) the Korean script, for which most layout corpora provide no coverage, and (b) the multi-page deck structure unique to card-news.

We make the problem concrete and reproducible with three contributions:

1. **A curated Korean card-news corpus** of 687 images in 66 ordered decks, with an automatic, high-recall labeling pipeline (OCR + line-to-block merging) that yields title/body boxes without manual annotation (Sec. 3).
2. **A layout system** with a fine-tuned YOLOv8 detector and two generation routes—an adapted DS-GAN baseline and a detection-driven *template engine*—which we compare for real-world quality (Sec. 4, 6).
3. **A sequential (time-series) analysis** of deck dynamics: we treat page index as a discrete time axis and quantify how visual structure evolves from cover to closing page (Sec. 5).

2 Related Work

Content-aware layout generation. LayoutGAN [2] and LayoutTransformer [3] generate element arrangements from noise or partial layouts. PosterLayout [1] conditions on a clean background and a saliency map and trains a domain-alignment GAN (DS-GAN) to place elements off salient regions; it is our generation baseline.

Object detection. We use YOLOv8 [4], the modern successor of the YOLO family [5], pre-trained on COCO [6] and fine-tuned for card elements.

Text detection / OCR. For bootstrap labels we use EasyOCR [7]; character-region methods such as CRAFT [8] are a higher-recall alternative for future work.

Saliency and inpainting. The generation route uses salient-object detection (U^2-Net [9], BASNet [10]) to find protected regions and large-mask inpainting (LaMa [11]) to erase existing text into clean backgrounds.

3 Dataset

Collection and standardization. We assembled 687 Korean cards: 109 seed references and 578 newly collected pages spanning public-sector, agriculture/smart-farm, and editorial themes. The 578 new pages are organized as 66 *decks* (ordered series of 4–11 pages). All images are re-encoded to RGB JPEG with ASCII filenames (so OpenCV/Ultralytics read them on any OS) and capped at 2048 px on the long side; the generation pipeline additionally rescales to the 513×750 PosterLayout canvas. Figure 1 shows samples.

Automatic labeling. Manual box annotation does not scale, so we generate *pseudo-labels*: EasyOCR [7] is

Figure 1: Representative pages from the 687-image, 66-deck Korean card-news corpus.

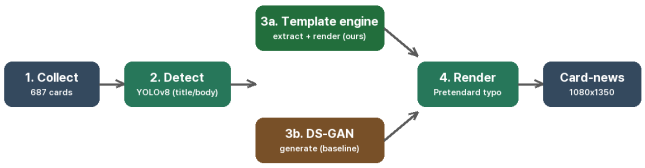

Figure 2: System overview. A shared detector feeds two generation routes; the template engine (3a) is our deployed path, DS-GAN (3b) is the research baseline.

run at high recall (`canvas_size`=2560, `mag_ratio`=2.0, `low_text`=0.3) to recover small body text; each text line is typed as *title* when its height exceeds 4.5% of the image height, else *body*. Because layout generation needs element *regions* rather than individual lines, we merge neighboring lines of the same class into **blocks** via a union-find over boxes expanded by a fraction of their height. This yields 284 title and 641 body boxes on the seed set; `logo`/`underlay` remain for manual annotation.

4 Method

Figure 2 summarizes the system: collect → detect → {template engine | DS-GAN} → render.

4.1 Element detection

We fine-tune YOLOv8-nano (and -small) from COCO weights on the pseudo-labels. Horizontal flipping is disabled (`fliplr`=0) because it mirrors Hangul; we use a card-tuned augmentation set (mild mosaic, HSV jitter, small scale/translate, random erasing). The detector both (a) supplies masks for inpainting and the `train_csv` used by DS-GAN, and (b) *extracts the layout templates* described next.

4.2 Layout generation: two routes

(3b) DS-GAN baseline. Following PosterLayout [1], we construct the required `Dataset/` (inpainted posters, dual saliency maps, `train_csv` of normalized element boxes) using U^2-Net/BASNet saliency and LaMa inpainting, and fine-tune DS-GAN to generate (class, box) layouts on clean Korean backgrounds. While the model produces plausible boxes, its *rendered* output is not deployable: it cannot place legible Korean glyphs and layout stability varies run-to-run.

(3a) Detection-driven template engine (ours). We instead *copy and re-render* real layouts. The detector localizes title/body blocks on a reference card; classical inpainting (or LaMa) removes the original text; new content is typeset back into the same block geometry with the Pretendard [12] typeface. We additionally maintain a small library of layout archetypes (editorial, centered, bottom-panel, cover) distilled from the corpus, and a saliency-aware placement step that selects the calmest image band (lowest mean gradient magnitude) for text and applies a luminance-adaptive scrim so the result is legible over *any* background.

4.3 Rendering

Cards are composited at 1080×1350 (Instagram 4:5). Text color is chosen from the background luminance under each block (white on dark, near-black on light), font size is auto-fit per box, and a soft rounded scrim guarantees contrast and covers residual texture. Typography uses Pretendard Bold for titles and Regular for body, with a fixed spacing system and accent rules.

5 Sequential Analysis of Decks

A card-news deck is an ordered sequence of pages; we treat the page index as a discrete time axis and ask how layout/visual structure evolves along it. Over the 66 decks (≥ 4 pages each) we compute, per page, four cv2 features—Canny *edge density* (text/graphic density), *brightness*, Hasler–Süsstrunk *colorfulness* [13], and HSV *saturation*—and average them into ten relative-position bins (0=cover, 1=last).

Figure 3 shows a consistent **cover-vs-interior** pattern: covers are notably *darker* (brightness 0.62 vs. $\approx$ 0.71 for interiors), more *saturated* (0.33 vs. $\approx$ 0.27) and more *colorful*, and have the *lowest edge density* (0.097). Interior pages are brighter and denser (more text), and the final page partly reverts toward the cover's saturated tone. This quantifies a design convention—an eye-catching, high-chroma cover followed by readable content pages—and motivates *position-conditioned* templates (Sec. 4): the engine selects a cover archetype for page 1 and content archetypes thereafter.

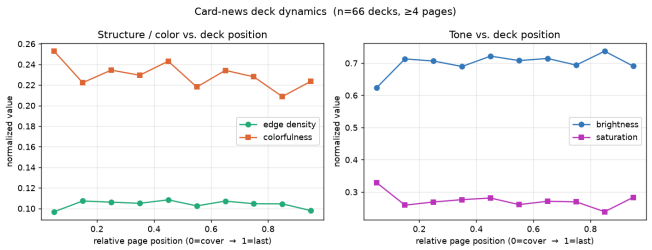

Figure 3: Deck dynamics across relative page position (66 decks). Covers are darker, more saturated/colorful and less edge-dense than interior pages.

Configuration	mAP@50–95	mAP@50
scratch (no pretrain)	0.502	0.736
baseline (n, freeze 10)	0.662	0.844
freeze 0	0.671	0.851
aug: heavy	0.644	0.838
aug: card-tuned	0.696	0.838
img 768	0.680	0.841
YOLOv8-s (freeze 10)	0.705	0.852
long300 + card aug (best)	**0.718**	**0.854**

Table 1: Detector ablation on the 109-image seed set (pseudo-labels).

6 Experiments

Setup. YOLOv8 fine-tuning, 640 px, 150–300 epochs, batch tuned to GPU; metrics are box mAP. The seed split is 93 train / 16 val. Cloud training uses an RTX 4090; development/inference run on an RTX 3050.

6.1 Detector ablation

Table 1 and Figure 4 report the ablation. Transfer learning is the dominant factor (scratch 0.502 vs. baseline 0.662). A card-tuned augmentation and a longer schedule give the best nano model (`long300_card`: mAP@50–95 = 0.718, mAP@50 = 0.854); YOLOv8-small is competitive (0.705). Seed repeats vary by < 0.02 mAP, indicating stable ranking. Qualitative detections are shown in Figure 5.

Robustness (5-fold CV). Five-fold cross-validation of the baseline gives mAP@50–95 $= 0.611 \pm 0.049$, confirming the small-data result is not split-specific.

Data scale (109 vs. 687). To measure the effect of the 578 new images we adopt a *leak-free* protocol: a common test set (15% of the new images) is held out and never trained on, and the best recipe is trained on (i) the 109 seed images and (ii) the full 687, with both evaluated on the identical test set. The tooling is released; cloud training of this comparison is in progress and we report it as a protocol rather than a completed result, to avoid drawing conclusions from unfinished runs.

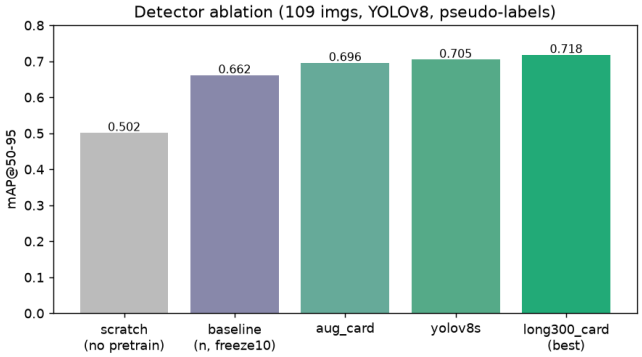

Figure 4: Key ablation results (box mAP@50–95). Transfer learning and a card-tuned, longer schedule drive the gains.

Figure 5: Detector predictions on held-out validation cards (title/body).

6.2 Generation quality

Figure 6 shows template-engine outputs and a copy-and-re-render example (original → cleaned background → recomposed). The template engine yields legible, consistently designed Korean cards at 1080 × 1350, whereas the DS-GAN route produces usable *boxes* but not deployable rendered text. For a production card-news service we therefore adopt the template engine and retain DS-GAN as a research comparison.

7 Discussion and Limitations

Pseudo-labels cap detector quality: EasyOCR misses very small or stylized text, so label recall—not data volume—is the binding constraint, addressed here by high-recall OCR settings

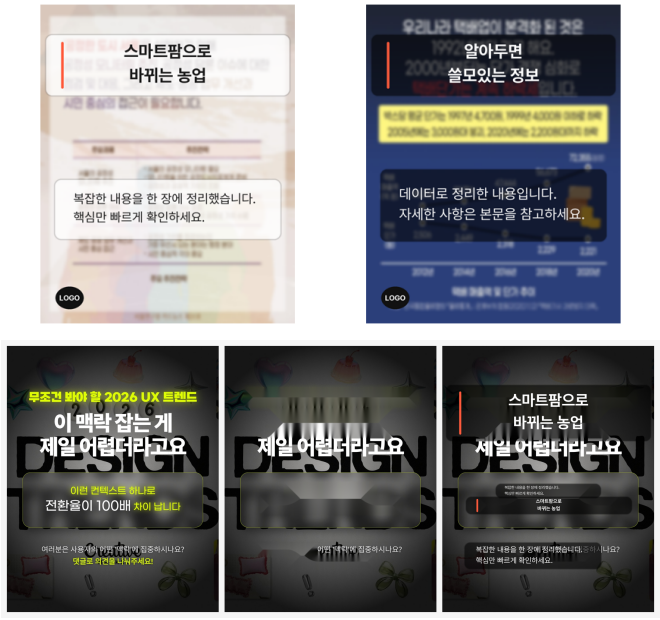

Figure 6: Top: template-engine cards (high-res, Pretendard, adaptive color). Bottom: layout copy—*original* | *inpainted background* | *recomposed* with new text in the analyzed layout.

and block merging, and in future by CRAFT [8] or manual `logo/underlay` annotation. The DS-GAN route is limited by Korean glyph rendering and small-data instability. The sequential analysis uses low-level proxies; detector-derived element statistics per page are a natural extension.

8 Conclusion

We presented a reproducible pipeline for Korean card-news layout: a curated 687-image, 66-deck corpus with automatic block labels, a fine-tuned YOLOv8 detector (mAP@50–95 $= 0.718$), two generation routes whose comparison favors a detection-driven template engine for deployment, and a time-series analysis that reveals a quantifiable cover-vs-interior design convention. The artifacts provide a basis for a practical Korean card-news generation service.

References

[1] H.-Y. Hsu, X. He, Y. Peng, H. Kong, and Q. Zhang, "Posterlayout: A new benchmark and approach for content-aware visual-textual presentation layout," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

[2] J. Li, J. Yang, A. Hertzmann, J. Zhang, and T. Xu, "Layoutgan: Generating graphic layouts with wireframe discriminators," in *International Conference on Learning Representations (ICLR)*, 2019.

[3] K. Gupta, J. Lazarow, A. Achille, L. Davis, V. Mahadevan, and A. Shrivastava, "Layouttransformer: Layout generation and completion with self-attention," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.

[4] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics yolov8." https://github.com/ultralytics/ultralytics, 2023.

[5] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[6] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft coco: Common objects in context," in *European Conference on Computer Vision (ECCV)*, 2014.

[7] JaidedAI, "Easyocr." https://github.com/JaidedAI/EasyOCR, 2020.

[8] Y. Baek, B. Lee, D. Han, S. Yun, and H. Lee, "Character region awareness for text detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

[9] X. Qin, Z. Zhang, C. Huang, M. Dehghan, O. R. Zaiane, and M. Jagersand, "U2-net: Going deeper with nested u-structure for salient object detection," *Pattern Recognition*, vol. 106, 2020.

[10] X. Qin, Z. Zhang, C. Huang, C. Gao, M. Dehghan, and M. Jagersand, "Basnet: Boundary-aware salient object detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

[11] R. Suvorov, E. Logacheva, A. Mashikhin, A. Remizova, A. Ashukha, A. Silvestrov, N. Kong, H. Goka, K. Park, and V. Lempitsky, "Resolution-robust large mask inpainting with fourier convolutions," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2022.

[12] H. Kil, "Pretendard: A variable, modern korean/latin sans-serif typeface." https://github.com/orioncactus/pretendard, 2021.

[13] D. Hasler and S. E. Suesstrunk, "Measuring colorfulness in natural images," in *Human Vision and Electronic Imaging VIII*, vol. 5007, SPIE, 2003.